

SEGURIDAD EN CÓMPUTO

Caso Práctico: Desarrollo de Aplicación Web con Gestión de Roles y Permisos de Usuario

*Nota de actualización: esta versión incorpora, respecto del documento original, el requisito de que la aplicación sea web, el consumo de una API REST protegida con JSON Web Tokens (JWT), y una nueva sección de requerimientos de seguridad computacional. Los cambios se distinguen en **negritas**.*

Objetivo del Caso Práctico:

Desarrollar **una aplicación web** que implemente un sistema de autenticación y autorización basado en roles y permisos, **consumiendo una API REST protegida mediante JSON Web Tokens (JWT)**. El objetivo es permitir que los usuarios accedan, desde el navegador, a diferentes funcionalidades dependiendo de su rol dentro del sistema, garantizando en todo momento la confidencialidad e integridad de la información.

Requerimientos Funcionales:

1. Gestión de Usuarios:

- Registrar usuarios nuevos con los campos: Nombre, Email, Contraseña.
- Iniciar sesión.
- Implementar el cambio de contraseña y recuperación de contraseña.

2. Gestión de Roles:

- Implementar tres roles predeterminados:
 - Administrador: Tiene acceso completo a todas las funcionalidades.
 - Editor: Puede crear, editar y eliminar contenidos, pero no puede gestionar usuarios ni configuraciones globales.
 - Usuario Regular: Solo puede ver y consumir contenido.

3. Gestión de Permisos:

- Los permisos deben ser específicos para cada acción dentro de la aplicación, como:
 - Lectura: Acceso a visualizar ciertos recursos.
 - Escritura: Acceso para crear o modificar recursos.
 - Eliminación: Permiso para eliminar recursos.
- Asignar permisos a los roles de forma dinámica, permitiendo que cada rol tenga diferentes permisos en distintas áreas de la aplicación.

4. Control de Acceso:

- Un usuario solo puede acceder a ciertas páginas o funcionalidades si tiene los permisos adecuados.
- Guardar el historial de acceso al sistema por parte de todos los usuarios.

5. Dashboard de Administración:

- Implementar un panel de administración donde los administradores puedan:
 - Ver todos los usuarios registrados.
 - Asignar o revocar roles a los usuarios.
 - Crear nuevos roles y asignar permisos a estos roles.
 - Consultar un registro de auditoría para ver las acciones realizadas por los usuarios.

6. Arquitectura Web y Consumo de Servicios: (Nuevo)

- La aplicación debe estar compuesta por un frontend web (accesible desde el navegador) y un backend que exponga una API REST.
- El frontend debe consumir todos los datos y funcionalidades exclusivamente a través de la API REST del backend (no acceso directo a la base de datos desde el cliente).
- Toda comunicación entre frontend y backend debe autenticarse mediante JSON Web Tokens (JWT), enviados en el encabezado Authorization.

Contenido de la aplicación:

- El contenido puede ser definido a partir de una aplicación que ya hayan desarrollado en proyectos pasados.

Requerimientos Técnicos:

1. El lenguaje de programación y el motor de base de datos (SMBD) pueden elegirse de acuerdo con sus conocimientos.
2. La aplicación debe ser una aplicación web, accesible desde un navegador (el frontend puede desarrollarse con la tecnología de su elección: HTML/CSS/JS, React, Angular, Vue, Blazor, etc.). (Nuevo)
3. El backend debe exponer sus funcionalidades como una API REST, utilizando los verbos HTTP correspondientes (GET, POST, PUT, DELETE) y respuestas en formato JSON. (Nuevo)
4. La autenticación y autorización entre el frontend y el backend debe implementarse mediante JSON Web Tokens (JWT): (Nuevo)
 - El inicio de sesión debe devolver un token firmado (JWT) una vez validadas las credenciales del usuario.
 - Cada petición hacia un endpoint protegido debe incluir el token en el encabezado Authorization: Bearer <token>.
 - El backend debe validar la firma, la expiración y los datos del token (rol y permisos) antes de procesar cualquier solicitud.

Requerimientos de Seguridad Computacional: (Nueva sección)

1. Cifrado de contraseñas: las contraseñas deben almacenarse utilizando un algoritmo de hash seguro con salt (bcrypt, Argon2 o PBKDF2); nunca en texto plano ni con algoritmos reversibles.
2. Comunicación cifrada (HTTPS/TLS): toda comunicación entre el cliente y el servidor, incluyendo el envío del JWT y de las credenciales, debe realizarse sobre HTTPS.
3. Gestión segura de tokens JWT:
 - Definir un tiempo de expiración corto para el access token (por ejemplo, 15-30 minutos) y utilizar refresh tokens para renovar la sesión sin solicitar credenciales nuevamente.
 - Firmar los tokens con un algoritmo seguro (HS256 con un secreto robusto, o RS256 con par de llaves) y mantener la clave/secreto fuera del código fuente (variables de entorno o un gestor de secretos).
 - Evitar almacenar el JWT en localStorage cuando sea posible, dado el riesgo de robo vía XSS; preferir cookies con los atributos HttpOnly, Secure y SameSite.
4. Autorización verificada en el servidor: cada endpoint de la API debe validar el rol y los permisos del usuario autenticado, independientemente de que la interfaz oculte opciones no permitidas.
5. Validación y sanitización de entradas: todos los datos recibidos (formularios, parámetros de URL, cuerpos JSON) deben validarse en el servidor para prevenir inyección SQL, XSS y otros ataques de manipulación de datos.

- 6. Protección contra ataques de fuerza bruta:** limitar los intentos fallidos de inicio de sesión (bloqueo temporal o CAPTCHA) y aplicar límites de solicitudes (rate limiting) sobre los endpoints de autenticación.
- 7. Registro de auditoría con trazabilidad:** el historial de acceso y las acciones administrativas deben registrar usuario, fecha y hora, dirección IP y acción realizada; dicho registro no debe poder modificarse desde la propia aplicación.
- 8. Manejo seguro de CORS:** la API debe configurarse para aceptar peticiones únicamente desde los orígenes (dominios) autorizados del frontend.
- 9. Principio de mínimo privilegio:** cada rol debe tener acceso únicamente a los recursos y acciones estrictamente necesarios para cumplir su función.
- 10. Manejo de errores sin fuga de información:** los mensajes de error mostrados al cliente no deben revelar detalles internos del sistema (trazas de pila, estructura de la base de datos, versiones de software, etc.).